CSE 4621: Machine Learning

Neural Networks

Lecture 6

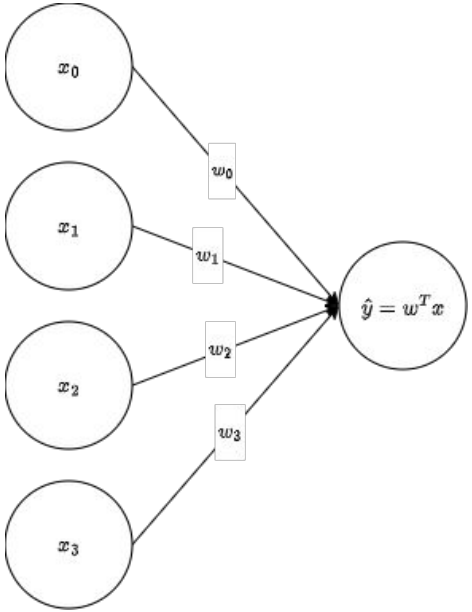
Ishmam Tashdeed
Lecturer, CSE IUT

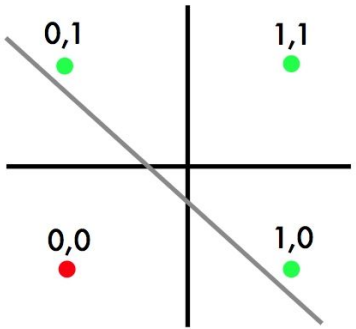
Lecture Outline

- Foundations
  - Perceptron
  - UAT
  - MLP
  - Activations
- Forward pass
  - Loss function
  - Compute graphs
- Back propagation
  - Chain rule
  - Full Derivation
- Optimization
  - SGD variants
  - LR scheduling
  - Dropout
  - Weight initialization
- Evaluation Metrics
  - Accuracy
  - Precision
  - Recall
  - F1 Score
  - ROC-AUC

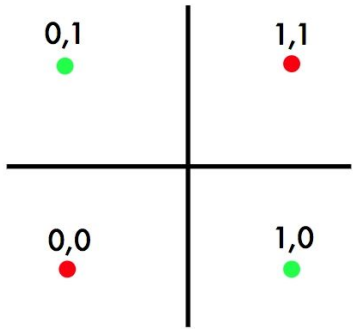
Perceptron

Perceptron is the building block of neural networks, designed by Frank Rosenblatt (1957). It could classify linearly separable problems





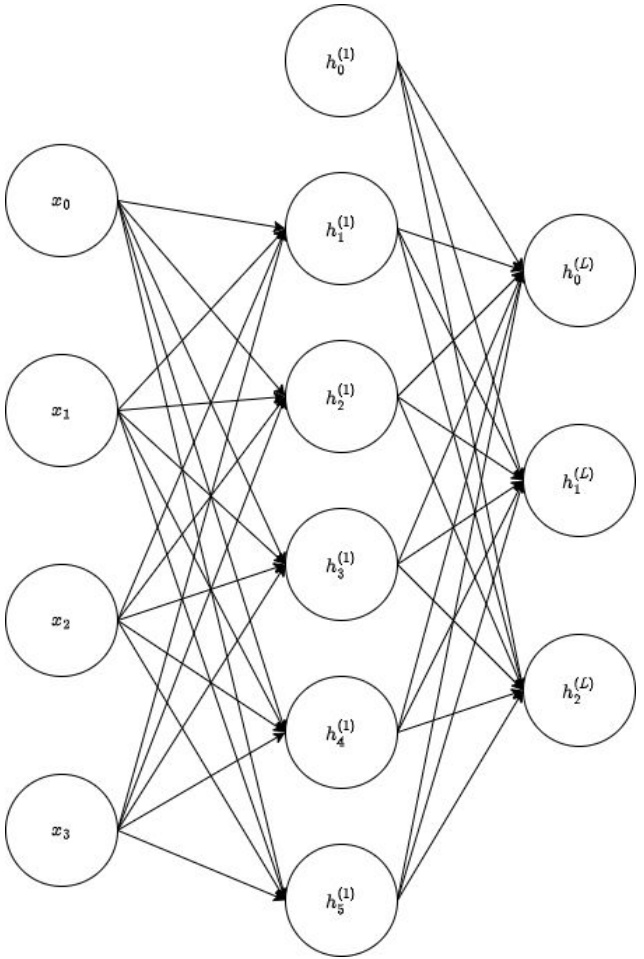
OR



XOR

Multi-Layer Perceptron (MLP)

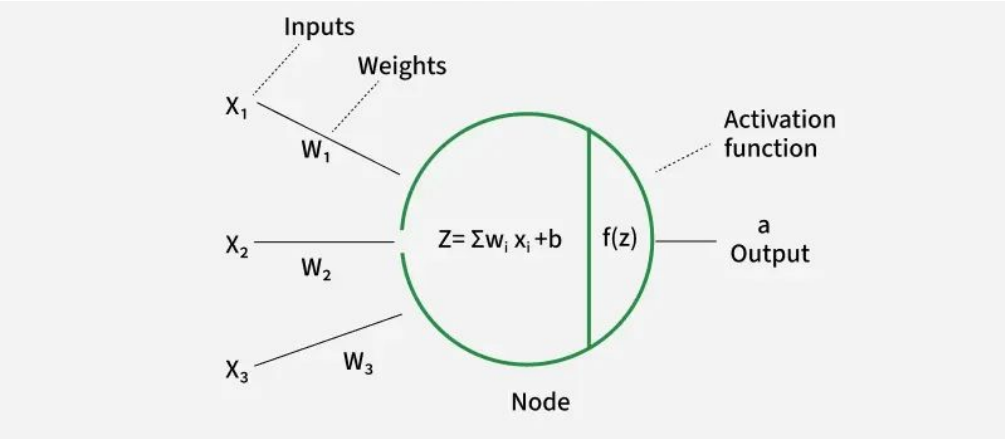
- Stack multiple perceptrons together
- Composition: input → hidden layers → output
- For each layer (l): $h^{(l)} = \sigma(W^{(l)} h^{(l-1)} + b^{(l)})$
- Parameters: $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}} \quad ; \quad b^{(l)} \in \mathbb{R}^{n_l}$
- Layer 0 = Input layer ; Layer L = Output layer
- Total parameters: $\sum_{l=0}^{L} (n_l \cdot n_{l-1} + n_l)$

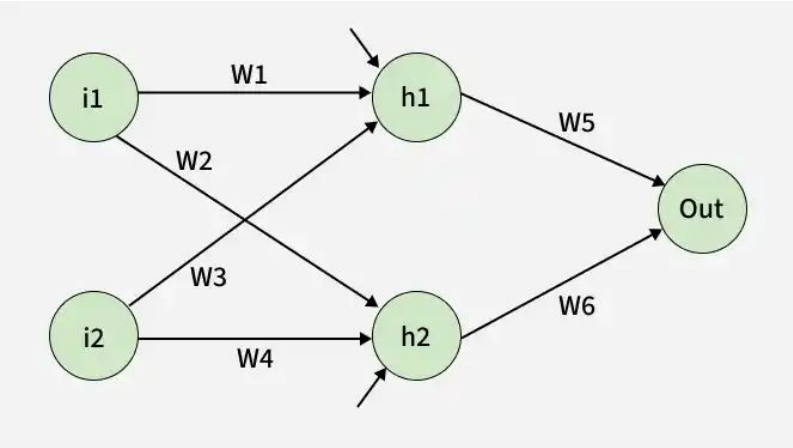


"Depth" = # of layers "Width" = # of perceptrons per layer

Activation Functions

Activation function is applied to the weighted sum of inputs to a neuron. It introduces non-linearity. What if we don't use any activation function?





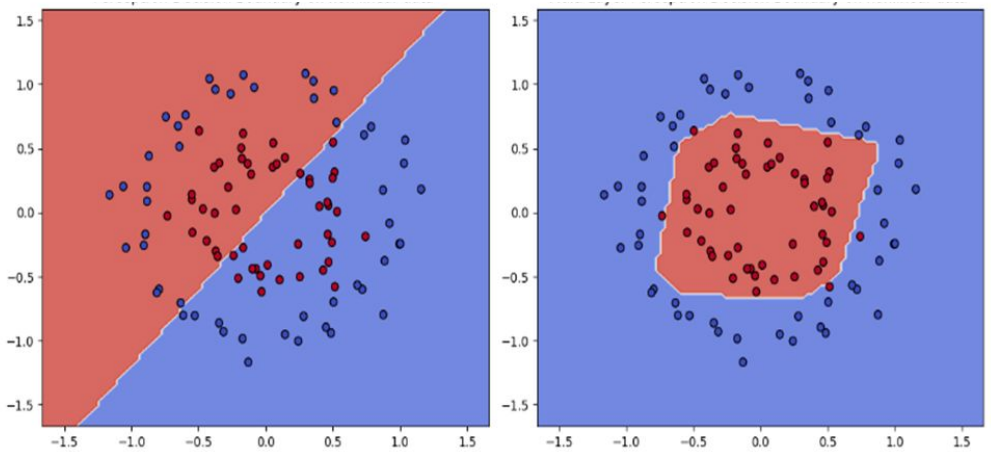
Activation Functions

Activation functions must be:

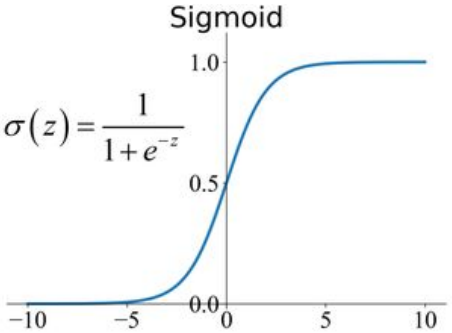
- Differentiable
- Fast to compute, zero-centered output

Popular activation functions:

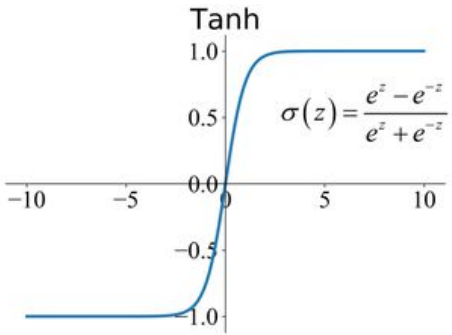
- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- GELU



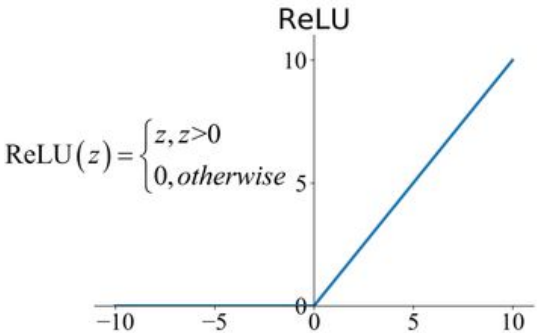
Activation Functions



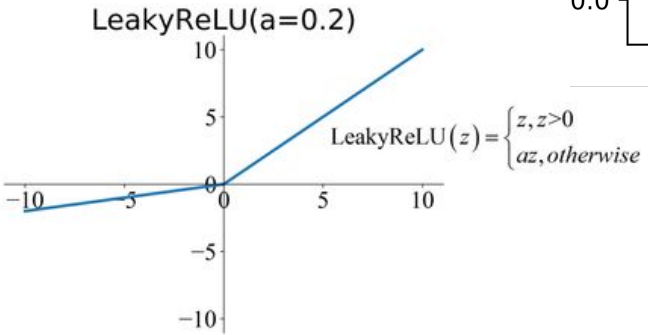
(a)



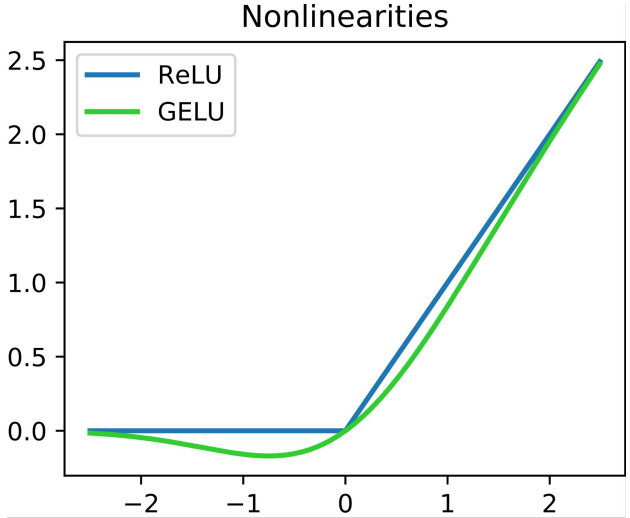
(b)



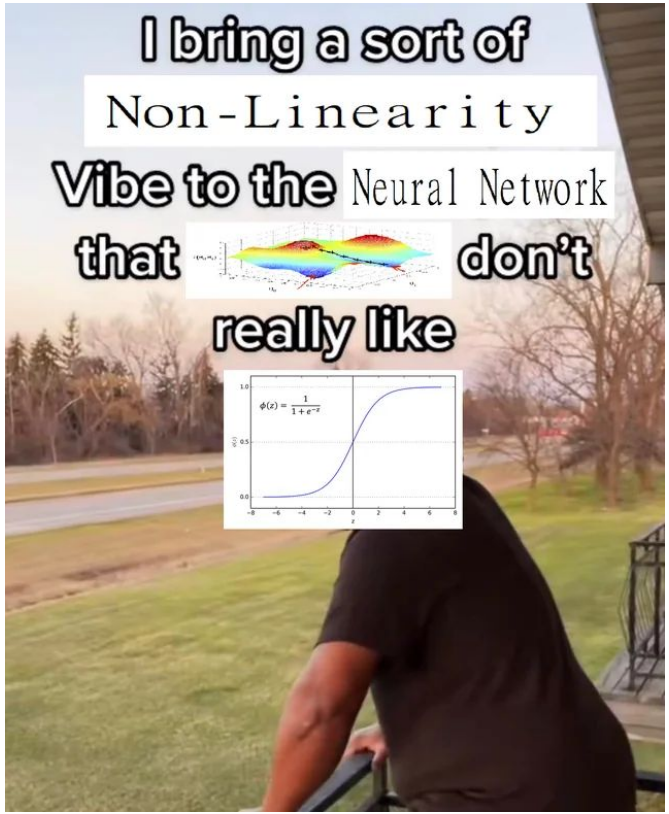
(c)



(d)



Activation Functions

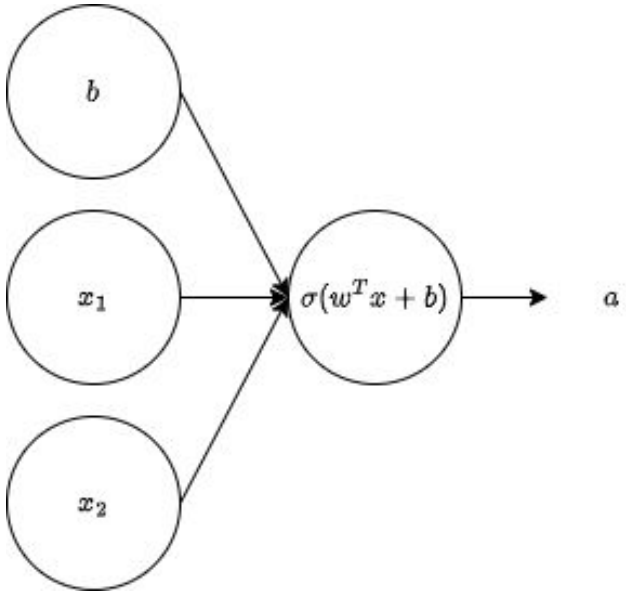


Forward Pass

A signal moves from the input layer through the hidden nodes and finally to the output layer. A forward pass is a sequence of matrix multiplications and activation functions.

$$z = w^T x + b$$

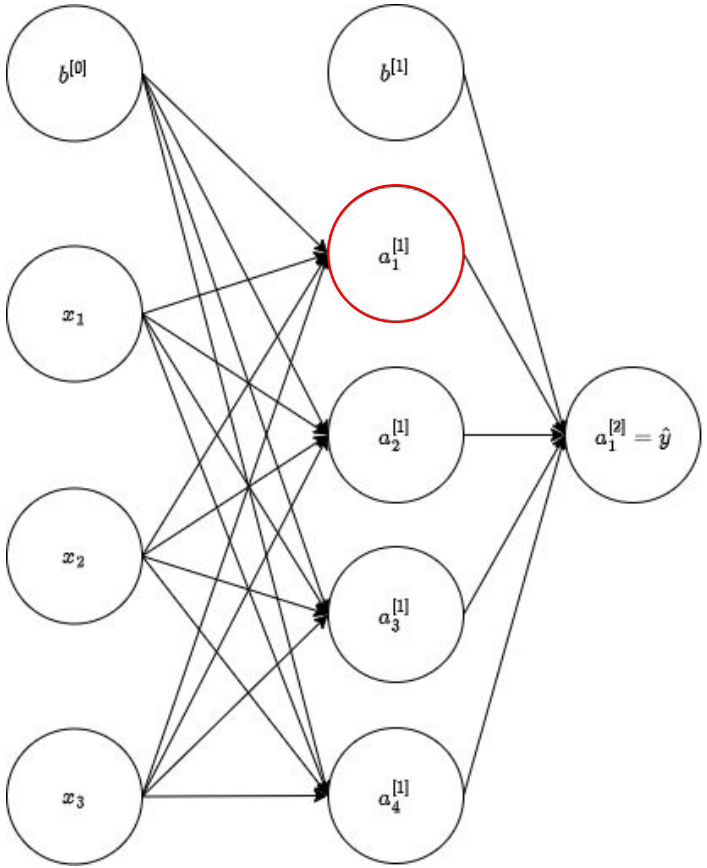
$$a = \sigma(z)$$



Forward Pass

$$z_1^{[1]} = {w_1^{[1]}}^T x + b_1^{[0]}$$

$$a_1^{[1]} = \sigma(z_1^{[1]})$$

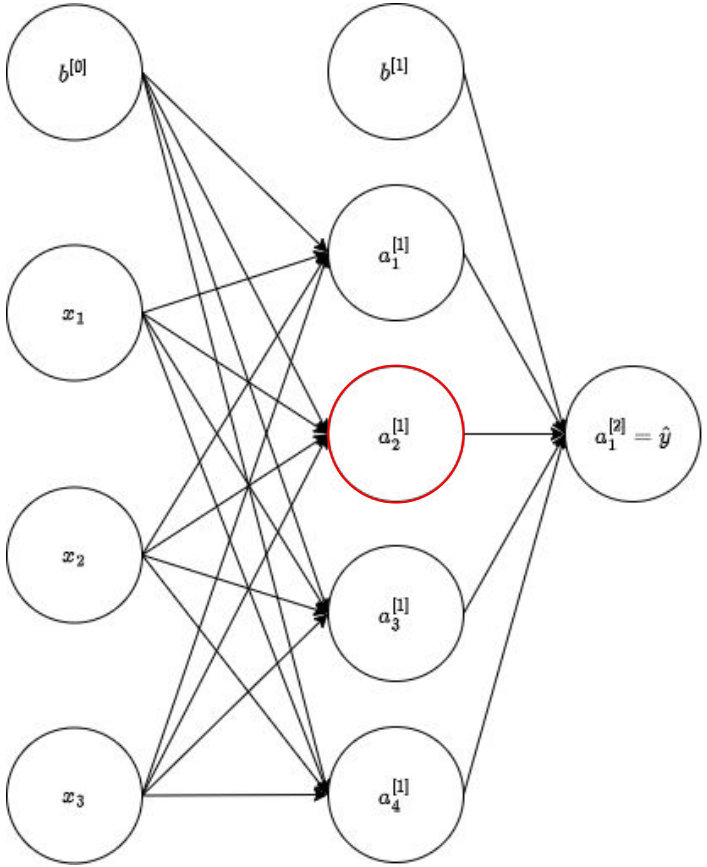


Forward Pass

$$z_1^{[1]} = {w_1^{[1]}}^T x + b_1^{[0]}$$
$$a_1^{[1]} = \sigma(z_1^{[1]})$$

$$z_2^{[1]} = {w_2^{[1]}}^T x + b_2^{[0]}$$
$$a_2^{[1]} = \sigma(z_2^{[1]})$$

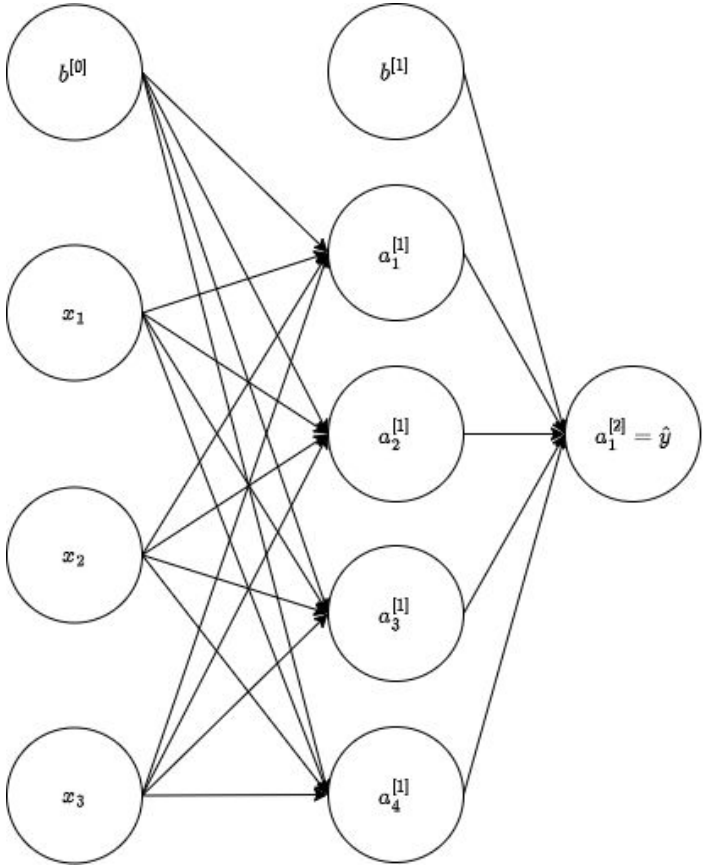
Let's combine all of these together.



Forward Pass

$$z^{[1]} = \begin{bmatrix} {w_1^{[1]}}^T \\ {w_2^{[1]}}^T \\ {w_3^{[1]}}^T \\ {w_4^{[1]}}^T \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[0]} \\ b_2^{[0]} \\ b_3^{[0]} \\ b_4^{[0]} \end{bmatrix}$$

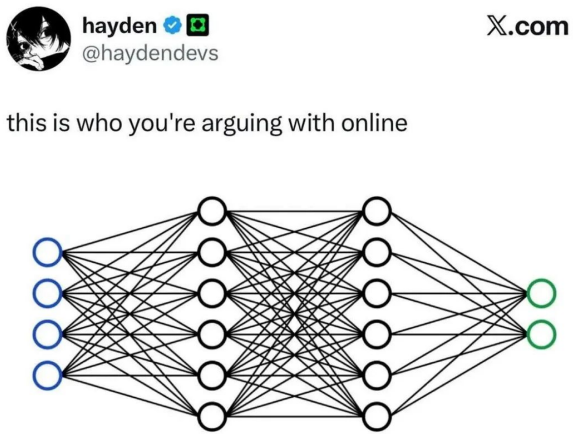
$$z^{[1]} = \begin{bmatrix} {w_1^{[1]}}^T x + b_1^{[0]} \\ {w_2^{[1]}}^T x + b_2^{[0]} \\ {w_3^{[1]}}^T x + b_3^{[0]} \\ {w_4^{[1]}}^T x + b_4^{[0]} \end{bmatrix} = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} \qquad a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix} = \begin{bmatrix} \sigma(z_1^{[1]}) \\ \sigma(z_2^{[1]}) \\ \sigma(z_3^{[1]}) \\ \sigma(z_4^{[1]}) \end{bmatrix}$$

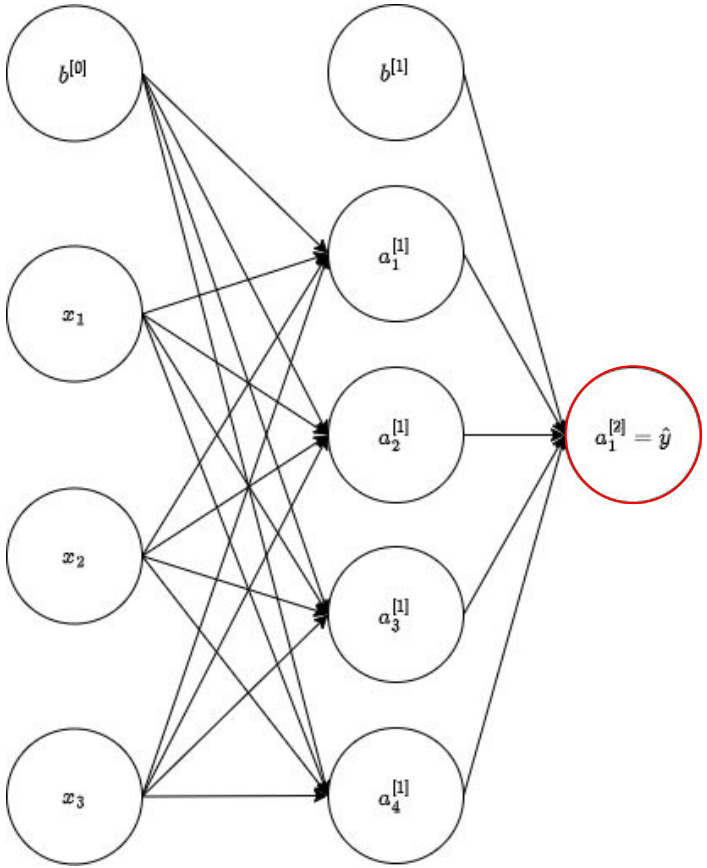


Forward Pass

$$z^{[2]} = W^{[2]} a^{[1]} + b^{[1]}$$
$$a^{[2]} = \sigma(z^{[2]})$$

This is the final output: $\hat{y}$





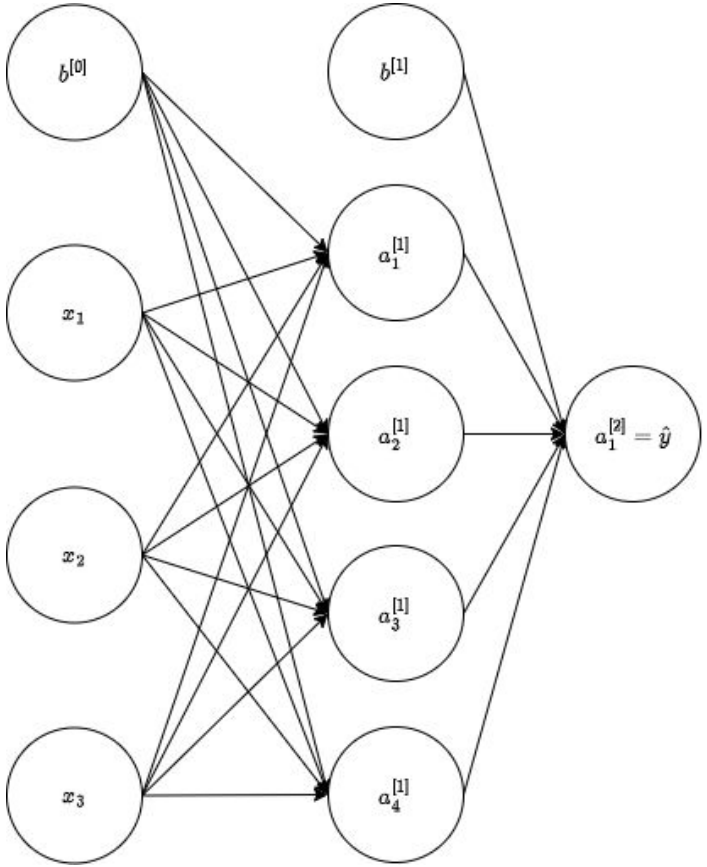
Forward Pass

Vectorizing across multiple examples:

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

$$A^{[1]} = \begin{bmatrix} | & | & & | \\ a^{[1](1)} & a^{[1](2)} & \cdots & a^{[1](m)} \\ | & | & & | \end{bmatrix}$$

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$
$$A^{[1]} = \sigma(Z^{[1]})$$
$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$
$$A^{[2]} = \sigma(Z^{[2]})$$



Cost Function

Notation:

m: Total number of samples in the dataset

L : Total number of layers

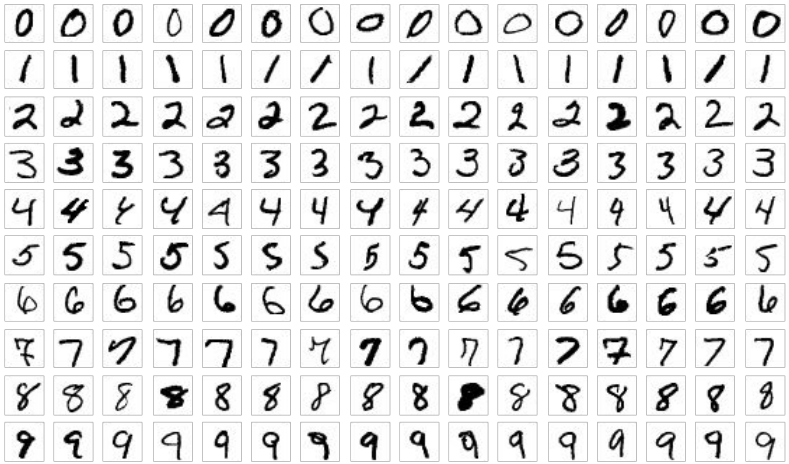
$n^{[l]}$: Total number of units in layer l (not counting bias)

For binary classification: $y = 0$ or 1 (Only one output neuron)

Multi-class classification: (K output neurons)

Cost Function

- A variation of one-vs-all
- We would have K neurons in the output layer
- $y^{(i)}$ would be one-hot vectors in $\mathbb{R}^K$



Cost Function

Cost for logistic regression with regularization:

$$J(W) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} log(\hat{y}^{(i)}) + (1 - y^{(i)}) log(1 - \hat{y}^{(i)})\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} W_j^2$$

For neural networks:

$$J(W,b) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\left[y_k^{(i)} \log\left(\hat{y}_k^{(i)}\right) + \left(1 - y_k^{(i)}\right) \log\left(1 - \hat{y}_k^{(i)}\right)\right]$$

$$J(W,b) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\left[y_k^{(i)} \log\left(\hat{y}_k^{(i)}\right) + \left(1 - y_k^{(i)}\right) \log\left(1 - \hat{y}_k^{(i)}\right)\right] + \frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{n^{[l]}}\sum_{j=1}^{n^{[l-1]}}\left(W_{ij}^{[l]}\right)^2$$

Gradient Descent

$$J(W,b) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\left[y_k^{(i)}\log\left(\hat{y}_k^{(i)}\right) + \left(1-y_k^{(i)}\right)\log\left(1-\hat{y}_k^{(i)}\right)\right] + \frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{n^{[l]}}\sum_{j=1}^{n^{[l-1]}}\left(W_{ij}^{[l]}\right)^2$$

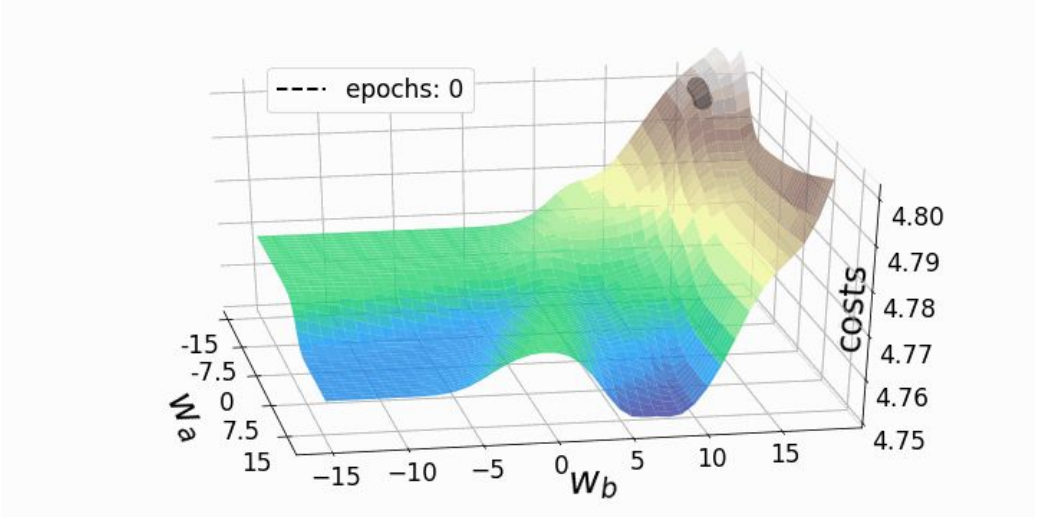
The objective is to minimize $J(W, b)$

```
repeat until convergence {
    for l = 1 ... L
```

$$W^{[l]} = W^{[l]} - \alpha\frac{\partial J(W,b)}{\partial W^{[l]}}$$

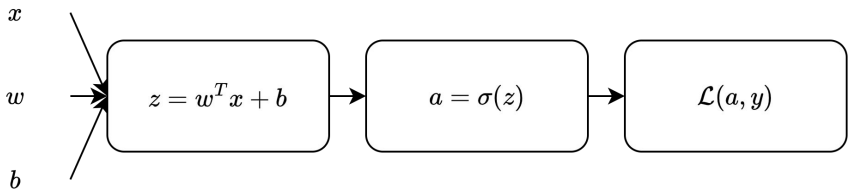
$$b^{[l]} = b^{[l]} - \alpha\frac{\partial J(W,b)}{\partial b^{[l]}}$$

```
}
```



Backpropagation

Assume logistic regression:



$$\mathcal{L}(a, y) = -y log(a) - (1 - y) log(1 - a)$$

$$da = \frac{d}{da}\mathcal{L}(a, y) = -\frac{y}{a} + \frac{1 - y}{1 - a}$$

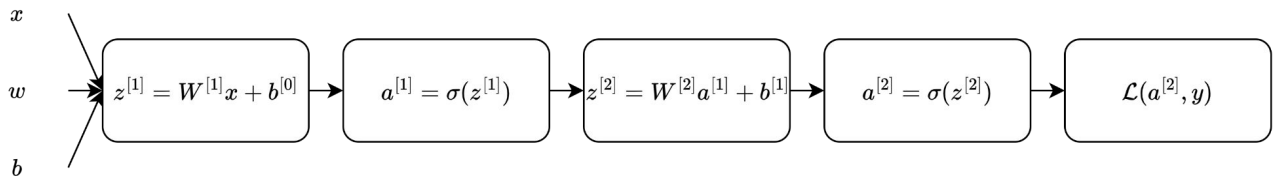
$$dz = \frac{d\mathcal{L}(a, y)}{dz} = \frac{d\mathcal{L}(a, y)}{da} \cdot \frac{da}{dz}$$

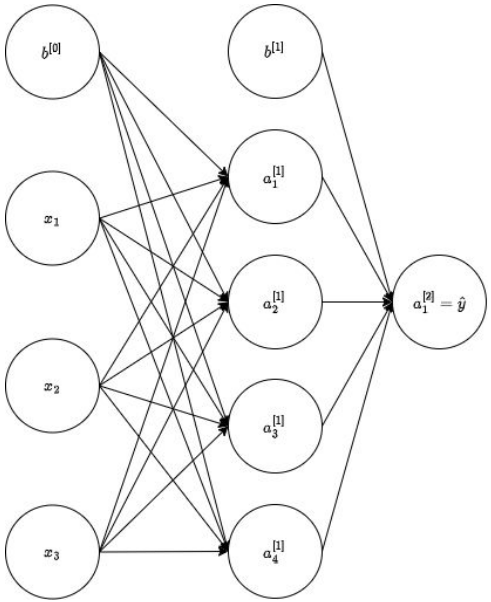
$$dz = da \cdot \sigma'(z)$$

$$dw = dz \cdot x$$

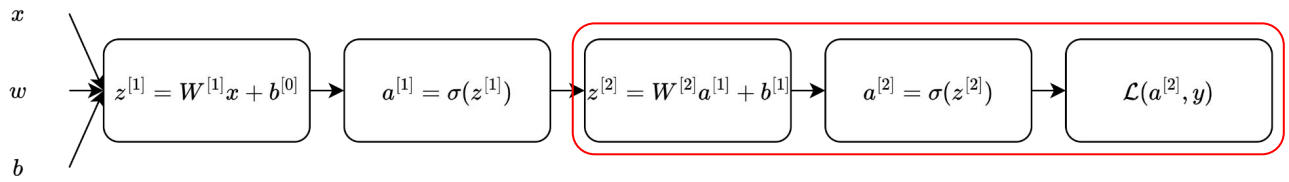
$$db = dz$$

Backpropagation





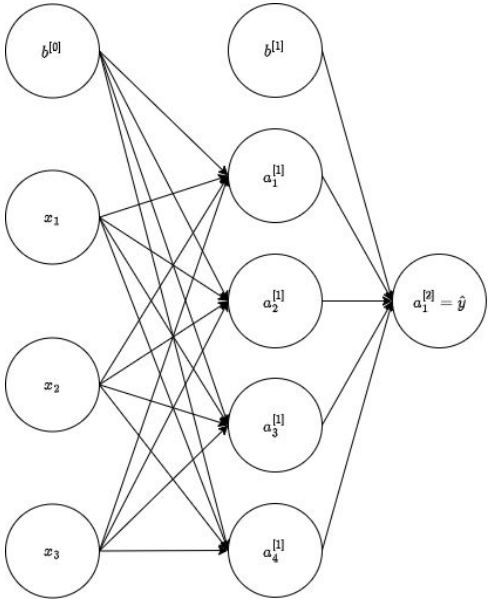
Backpropagation



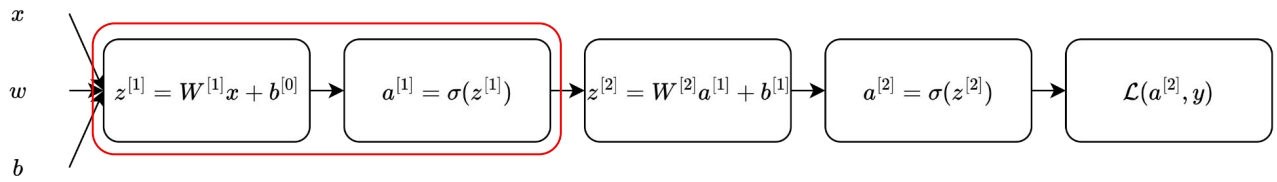
$$dz^{[2]} = a^{[2]} - y$$

$$dW^{[2]} = dz^{[2]}a^{[1]^T}$$

$$db^{[2]} = dz^{[2]}$$



Backpropagation



$$dz^{[2]} = a^{[2]} - y$$

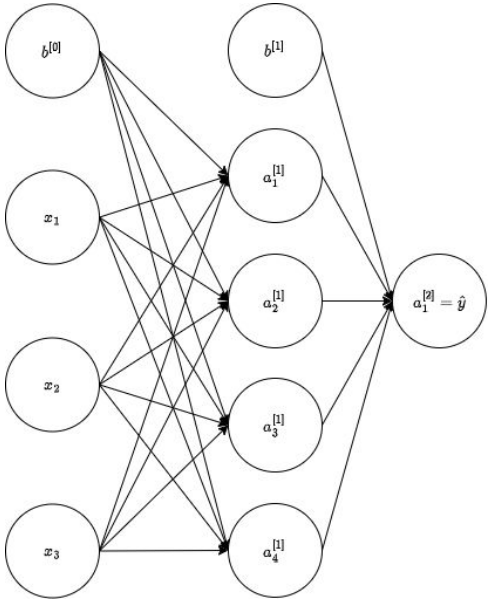
$$dW^{[2]} = dz^{[2]}a^{[1]^T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = W^{[2]^T}dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]}x^T$$

$$db^{[1]} = dz^{[1]}$$



Backpropagation

$$dz^{[2]} = a^{[2]} - y$$

$$dW^{[2]} = dz^{[2]} a^{[1]^T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = W^{[2]^T} dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

$$dZ^{[2]} = A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]^T}$$

$$db^{[2]} = \frac{1}{m} \sum dZ^{[2]}$$

$$dZ^{[1]} = W^{[2]^T} dZ^{[2]} * g^{[1]'}(Z^{[1]})$$

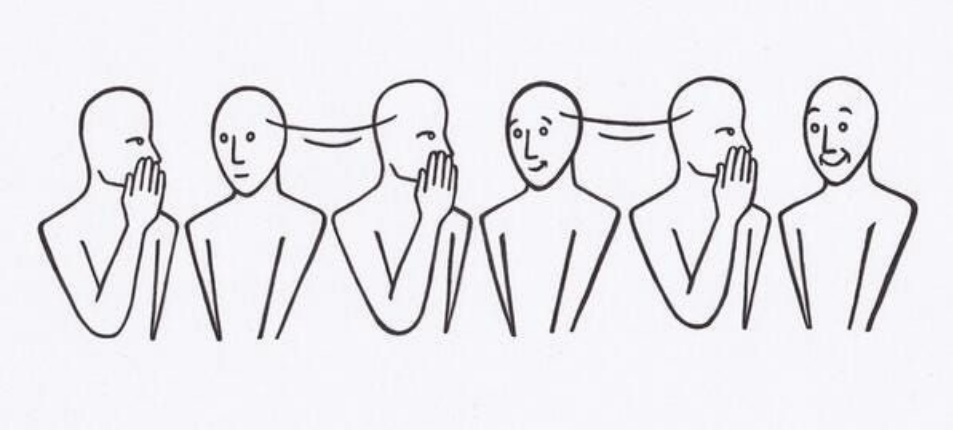
$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} \sum dZ^{[1]}$$

Recommended Resources: [3, 4]

Vanishing and Exploding Gradients

Deep networks learn by sending error signals backward. *Vanishing gradients* mean the signal fades away before reaching the early layers



Vanishing and Exploding Gradients

For the sigmoid function, its derivative is: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

The largest possible value for this derivative is: 0.25

If a neural network is 20 layers deep, a signal starting with 0.25 can get degraded by the time it reaches the earlier layers by:

$0.25^{20} = 9.094947 \times 10^{-13}$

Vanishing and Exploding Gradients

Exploding gradients are a problem where large error gradients accumulate and result in very large updates to neural network model weights during training.

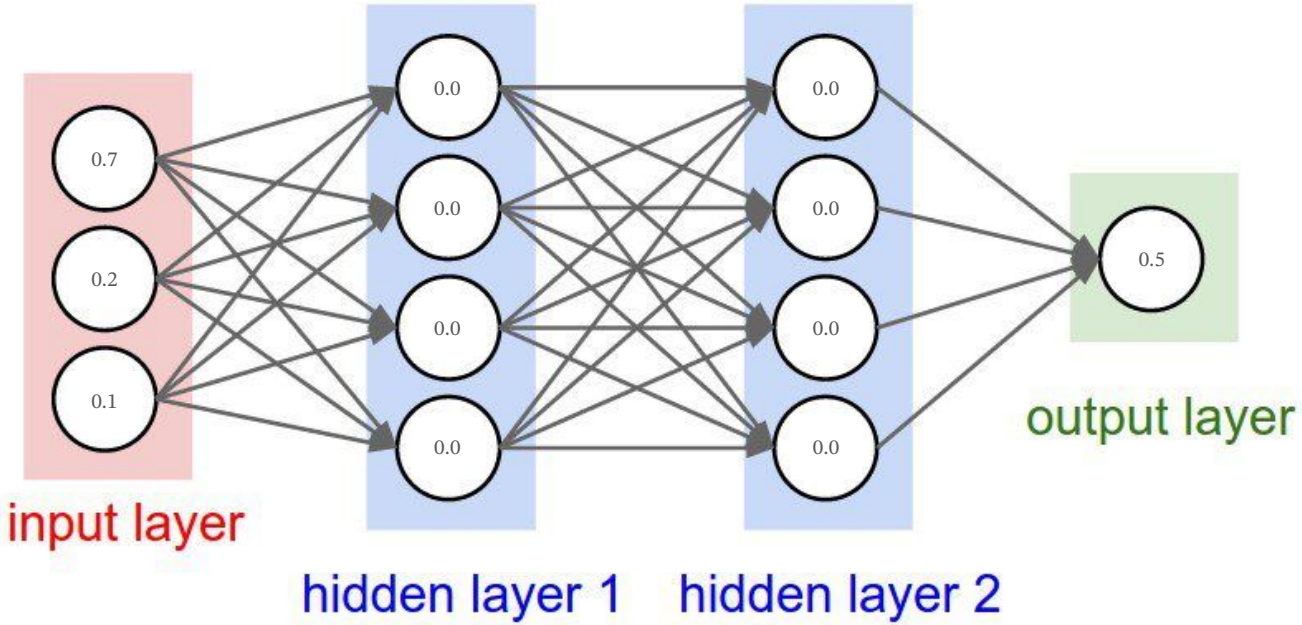
How to mitigate this:

- Redesign your network (Residual Connections)
- Clip the gradients
- Proper initialization
- Normalization layers

Recommended Resources: [5]

Weight Initialization

What happens if we initialize all the weights to 0?



Weight Initialization

How do we choose the initial random weights so that signals neither explode nor vanish as they pass through many layers?

Assume after some random initialization:

- Wights have a mean of 0 and a variance of 1
- Inputs also have a mean of 0 and a variance of 1

Thus, $Var(z) = n_{in} Var(w) Var(x)$

Since, there are *n* terms, $Var(z) = n$

Weight Initialization

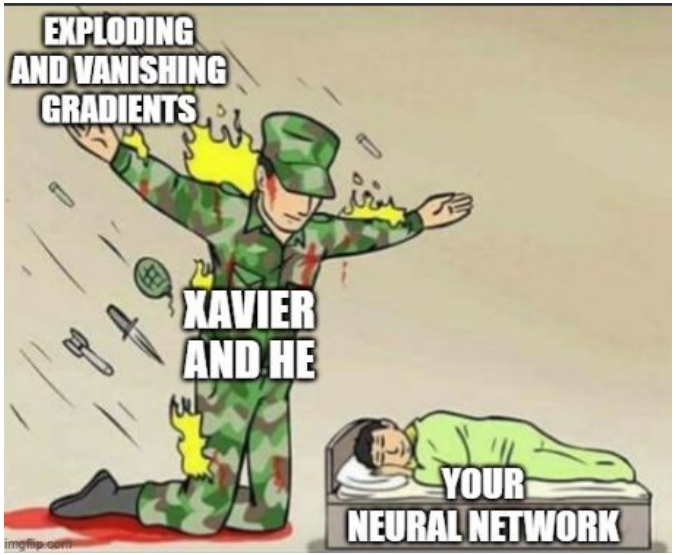
Ideally, every layer should preserve the scale of the signal: $Var(a_{l+1}) \approx Var(a_l)$

Xavier initialization (sigmoid, tanh)

- Choose weights with: $w \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$

He initialization (ReLU, Leaky ReLU, GELU)

- Choose weights with: $w \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$

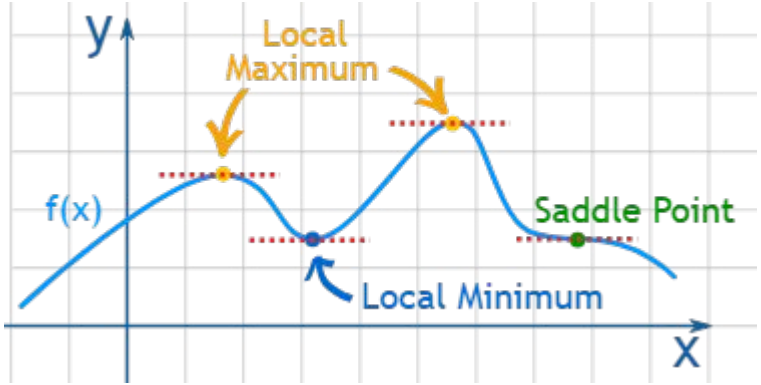


Optimization

Vanilla gradient descent updates weights such as: $w_{t+1} = w_t - \alpha \frac{\partial \mathcal{L}(w_t)}{\partial w_t}$

Each step only looks at the current gradient:

- slow in flat directions
- oscillates in steep directions
- gets stuck in saddle points



As neural networks have non-convex cost function, it can easily get stuck in a local minima!

SGD with Momentum

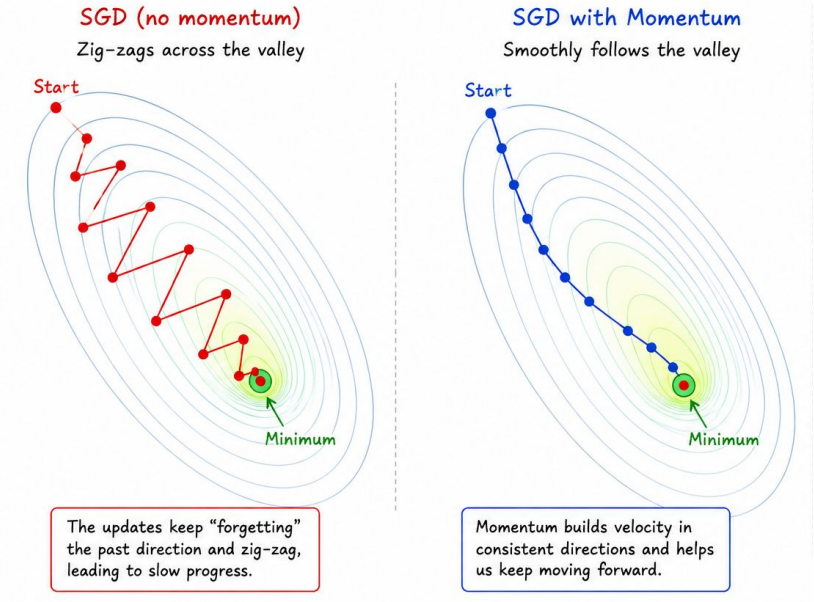
We maintain a velocity (v_t)

We update this velocity: $v_{t+1} = \beta v_t - \alpha \frac{\partial \mathcal{L}(w_t)}{\partial w_t}$

Then we update the weights as: $w_{t+1} = w_t + v_{t+1}$

So the optimizer remembers its recent direction and provides a smoother update

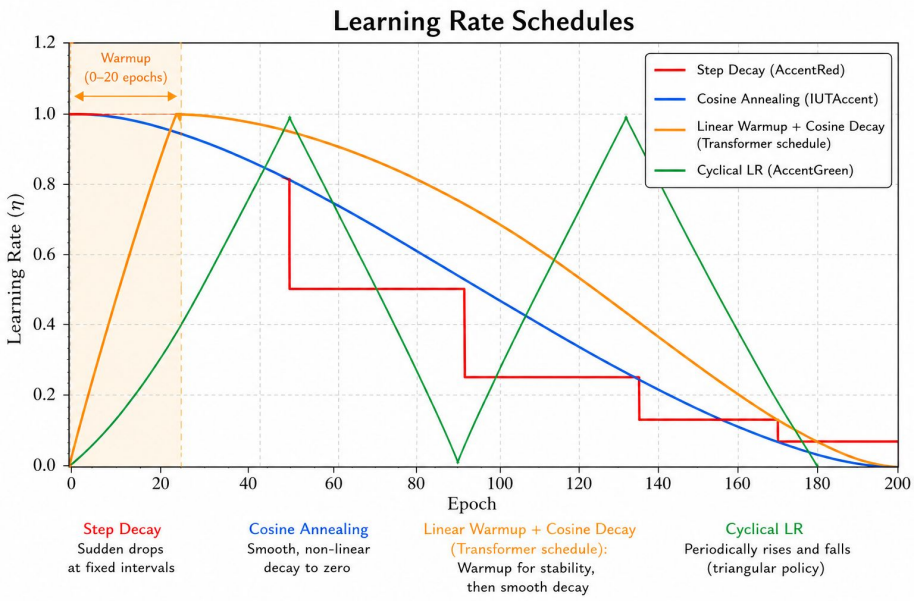
There are other methods such as: RMSProp, Adam, AdamW



Learning Rate Scheduling

Why schedule LR?

- Large LR early: fast progress.
- Small LR late: fine-grained convergence into minimum.
- Constant LR: suboptimal for both phases.

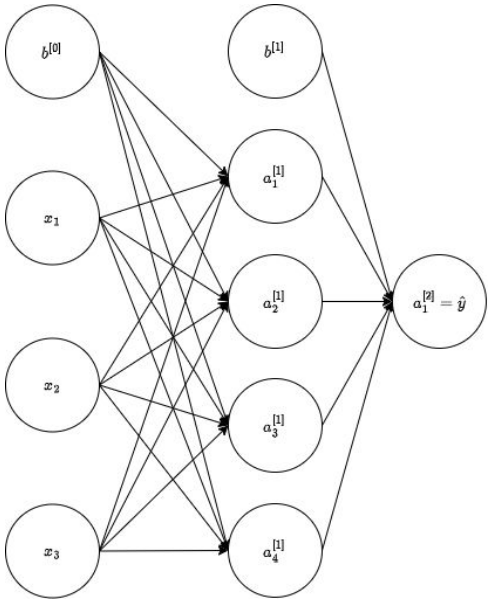


Regularization

L1 or L2 regularization works on neural networks, but what if the network relies too heavily on a single neuron?

Imagine a single neuron detecting some highly important feature. Many downstream neurons become highly dependent on it.

The network is said to become *co-adapted* → relying heavily on a fragile pattern

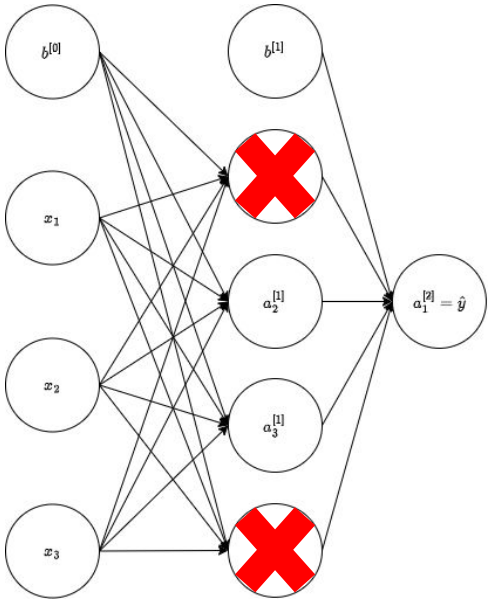


Dropout

During training, dropout randomly removes neurons:

- Each neuron is kept with probability p
- Dropped neurons contribute nothing in that forward / backward pass
- A slightly different sub-network is trained for every mini-batch

So dropout forces the network to learn representations that do not depend heavily on any single neuron

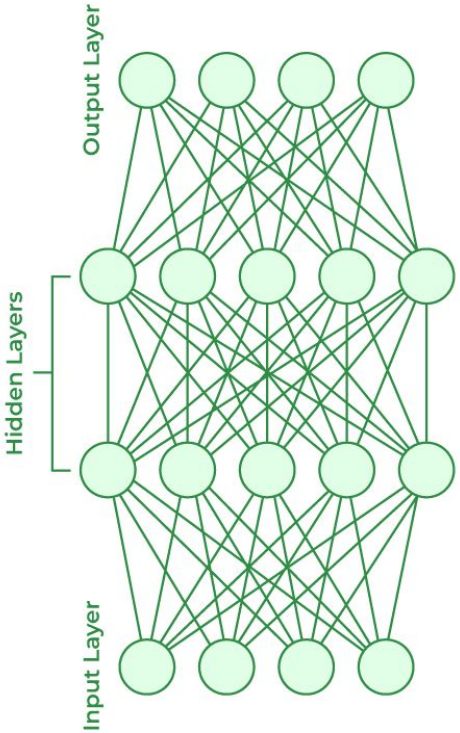


Batch Normalization

Imagine a simple neural network:

- The input to layer 1 changes with every sample, which in turn changes the input to layer 2, which changes the input to the final layer
- The distribution of input signals at each layers changes for every sample

This change in input distributions is called the: *Internal Covariate Shift*



Batch Normalization

For a mini-batch: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), (x^{(3)}, y^{(3)}), \ldots\}$

Compute the batch mean and variance for all inputs and normalize each sample as: $\hat{x}_i = \frac{x_i - \mu_b}{\sqrt{\sigma^2 + \epsilon}}$

Now activations will have:

- Mean = 0
- Variance = 1

But what if we needed that distribution?

Batch Normalization

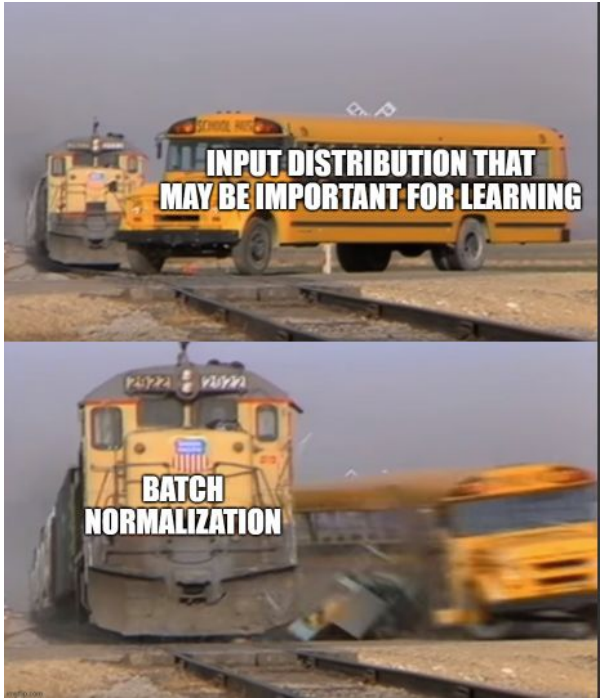
After normalizing, the batch normalization function learns:

$$y_i = \gamma \hat{x}_i + \beta$$

These parameters allow the network to recover any distribution that is optimal for the learning outcome

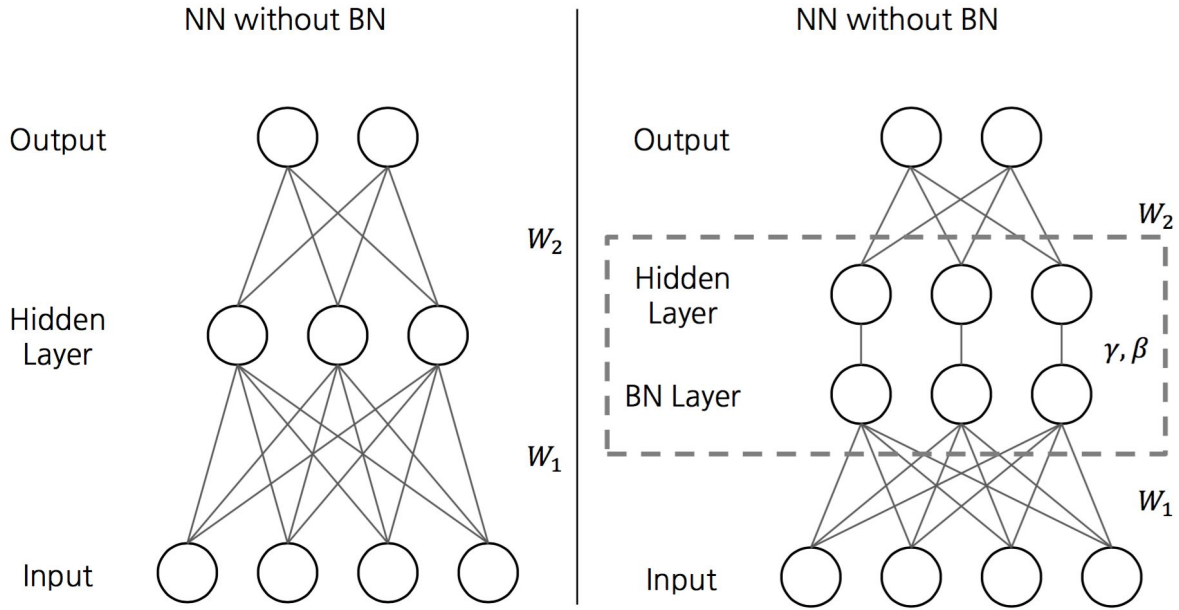
Because of γ and β, batch normalization can learn any mean and any variance (it is not restricted to mean=0 and variance=1)

This makes optimization easier by making the loss surface smoother



Batch Normalization

What about inference?



Batch Normalization

There are many variants of this:

- Batch Normalization → Normalizes across the mini-batch (Seen on CNNs)
- Layer Normalization → Normalize across the features of a single sample (Seen on Transformers)
- Instance Normalization → Similar to layer normalization, only normalizes a single channel (Seen on style transfer tasks)
- Group Normalization → Channels are normalized in groups (Seen on CV applications when GPU memory is limited)

Debugging Neural Networks

Loss not decreasing:

- Check learning rate
- Check data normalization
- Check if the labels are correct
- Try to overfit on a single sample/batch first

Loss becomes `Nan`:

- Exploding gradient → add gradient clipping
- Bad weight initialization → use He/Xavier

Debugging Neural Networks

Validation loss increases while training loss decreases:

- Add dropout, L2 regularization, reduce the number of hidden layers/neurons
- Check for data leakage

Training is very slow:

- Vanishing gradients → Switch activations to ReLU, add batch normalization
- Learning rate too small
- Bad weight initialization → use He/Xavier

Textbook Chapters

- [Goodfellow] - Chapter 6, 7, 8
- [Bishop] - Chapter 5.1-5.4
- [Alpaydin] - Chapter 11

Recommended Resources

1. Study urges caution when comparing neural networks to the brain
2. Neural Network Playground
3. Neural Networks Playlist | 3Blue1Brown
4. Backpropagation Paper
5. Sigmoid Vanishing Gradient

Thank you